UNIVERSITATEA TEHNICĂ A MOLDOVEI · FCIM / FAF

Operating Systems · anul universitar 2026–2027

Referat — Laborator 2

Meet the OS You Will Build
Track B · construiește propriul mini-OS (xv6)

Student:	Dulgheru Ion
Grupa:	FAF-241
Sistem folosit:	Ubuntu (nativ), host `ion-vitius`
Mediu de rulare:	QEMU, emulare RISC-V (`riscv64gc`)
Data:	28 septembrie 2026

Pregătirea mediului

Sistemul folosit e Ubuntu nativ (nu VM-ul de curs), deci toolchain-ul RISC-V și QEMU nu erau instalate implicit și au fost adăugate manual.

```
$ sudo apt install -y git build-essential gdb-multiarch qemu-system-misc \
    gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

La prima rulare a lui `make qemu`, emulatorul `qemu-system-riscv64` nu a fost găsit — pe această versiune de Ubuntu binarul a fost mutat într-un pachet separat, `qemu-system-riscv` (nu mai e inclus în `qemu-system-misc`). Fix:

```
$ sudo apt install qemu-system-riscv
$ hash -r
$ qemu-system-riscv64 --version
QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
```

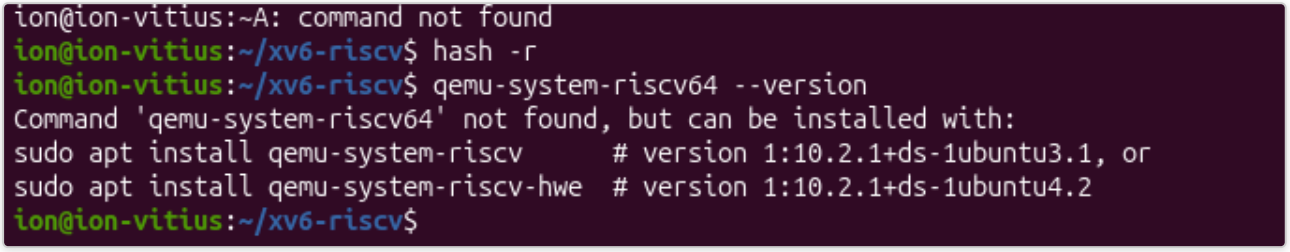


Fig. 1 — Diagnosticarea și corectarea pachetului lipsă pentru `qemu-system-riscv64`.

Part 1 · Build it and boot it

Clonarea sursei xv6 și compilarea kernelului + programelor din `user/`:

```
$ git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
$ cd ~/xv6-riscv
$ make qemu
```

După compilare, `make` a construit imaginea de disc (`mkfs`) și a pornit kernelul în QEMU. A apărut promptul `$`, confirmând un boot reușit:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

```
ion@ion-vitius:~/xv6-riscv$ hash -r
ion@ion-vitius:~/xv6-riscv$ qemu-system-riscv64 --version
Command 'qemu-system-riscv64' not found, but can be installed with:
sudo apt install qemu-system-riscv      # version 1:10.2.1+ds-1ubuntu3.1, or
sudo apt install qemu-system-riscv-hwe  # version 1:10.2.1+ds-1ubuntu4.2
ion@ion-vitius:~/xv6-riscv$ sudo apt install qemu-system-riscv
Installing:
  qemu-system-riscv

Installing dependencies:
  opensbi  qemu-efi-riscv64

Suggested packages:
  samba  vde2  passt

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 5
  Download size: 10.3 MB
  Space needed: 105 MB / 199 GB available

Continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu resolute/main amd64 opensbi all 1.8.1-1 [231 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 qemu-efi-riscv64 all 2025.11-3ubuntu7.2 [1,494 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 qemu-system-riscv amd64 1:10.2.1+ds-1ubuntu3.2 [8,5
86 kB]
Fetched 10.3 MB in 4s (2,452 kB/s)
Selecting previously unselected package opensbi.
(Reading database… 160530 files and directories currently installed.)
Preparing to unpack …/opensbi_1.8.1-1_all.deb…
Unpacking opensbi (1.8.1-1)…
Selecting previously unselected package qemu-efi-riscv64.
Preparing to unpack …/qemu-efi-riscv64_2025.11-3ubuntu7.2_all.deb…
Unpacking qemu-efi-riscv64 (2025.11-3ubuntu7.2)…
Selecting previously unselected package qemu-system-riscv.
Preparing to unpack …/qemu-system-riscv_1%3a10.2.1+ds-1ubuntu3.2_amd64.deb…
Unpacking qemu-system-riscv (1:10.2.1+ds-1ubuntu3.2)…
Setting up qemu-efi-riscv64 (2025.11-3ubuntu7.2)…
Setting up opensbi (1.8.1-1)…
Setting up qemu-system-riscv (1:10.2.1+ds-1ubuntu3.2)…
Processing triggers for man-db (2.13.1-1build1)…
ion@ion-vitius:~/xv6-riscv$ hash -r
qemu-system-riscv64 --version
QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
Copyright (c) 2003-2025 Fabrice Bellard and the QEMU Project developers
ion@ion-vitius:~/xv6-riscv$ cd ~/xv6-riscv
make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force
-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

Fig. 2 — Instalarea pachetului corect, compilarea kernelului (`riscv64-linux-gnu-gcc`) și boot-ul reușit al xv6 în QEMU.

Part 2 · Use xv6 as the Unix it is

```
$ ls
$ cat README
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
$ wc README
48 336 2441 README
```

(1) Trei programe din xv6

`cat` — afișează conținutul unui fișier (sau al stdin-ului); `echo` — afișează argumentele primite; `wc` — numără linii, cuvinte și octeți dintr-un fișier.

(2) Ce trebuie să existe în OS ca un pipe (|) să funcționeze?

Două lucruri: **procese separate** — shell-ul face `fork()` pentru fiecare comandă din pipeline, deci `ls` și `grep c` rulează ca două procese independente; și un **mecanism de comunicare inter-proces** — syscall-ul `pipe()` creează un canal cu două capete, iar shell-ul leagă ieșirea unui proces de intrarea celuilalt.

(3) Comparație: shell-ul xv6 vs shell-ul Linux (Lab 1)

Shell-ul xv6 (`sh.c`) e un subset minimal, de câteva sute de linii, care înțelege comenzi simple, redirectări și pipe-uri de bază; shell-ul Linux (bash) adaugă job control, istoric, autocomplete și scripting complex, dar ambele se bazează pe aceleași primitive de kernel: `fork` , `exec` , `pipe` .

```
ACKNOWLEDGMENTS

xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
to Peer Communications; ISBN: 1-57398-013-7; 1st edition (June 14,
2000)).  See also https://pdos.csail.mit.edu/6.1810/, which provides
pointers to on-line resources for v6.

The following people have made contributions: Russ Cox (context switching,
locking), Cliff Frey (MP), Xiao Yu (MP), Nickolai Zeldovich, and Austin
Clements.

We are also grateful for the bug reports and patches contributed by
Abhinavpatel00, Takahiro Aoyagi, Marcelo Arroyo, Hirbod Behnam, Silas
Boyd-Wickizer, Anton Burtsev, carlclone, Ian Chen, clivezeng, Dan
Cross, Cody Cutler, Mike CAT, Tej Chajed, Asami Doi,Wenyang Duan,
echtwerner, eyalz800, Nelson Elhage, Saar Ettinger, Alice Ferrazzi,
Nathaniel Filardo, flespark, Peter Froehlich, Yakir Goaron, Shivam
Handa, Matt Harvey, Bryan Henry, jaichenhengjie, Jim Huang, Matúš
Jókay, John Jolly, Alexander Kapshuk, Anders Kaseorg, kehao95,
Wolfgang Keller, Jungwoo Kim, Jonathan Kimmitt, Eddie Kohler, Vadim
Kolontsov, Austin Liew, Bruce Lowekamp, l0stman, Pavan Maddamsetti,
Imbar Marinescu, Yandong Mao, Matan Shabtay, Hitoshi Mitake, Carmi
Merimovich, mes900903, Mark Morrissey, mtasm, Joel Nider, Hayato Ohhashi,
OptimisticSide, papparapa, phosphagos, Harry Porter, Greg Price, Zheng
qhuo, Quancheng, RayAndrew, Jude Rich, segfault, Ayan Shafqat, Eldar
Sehayek, Yongming Shen, Fumiya Shigemitsu, snoire, Taojie, Cam Tenny,
tyfkda, Warren Toomey, Stephen Tu, Alissa Tung, Rafael Ubal, unicornx,
Amane Uehara, Pablo Ventura, Luc Videau, Xi Wang, WaheedHafez, Keiichi
Watanabe, Lucas Wolf, Nicolas Wolovick, wxdao, Grant Wu, x653, Andy
Zhang, Jindong Zhang, Icenowy Zheng, ZhUyU1997, and Zou Chang Wei.

ERROR REPORTS

Please send errors and suggestions to Frans Kaashoek and Robert Morris
(kaashoek,rtm@mit.edu).  The main purpose of xv6 is as a teaching
operating system for MIT's 6.1810, so we are more interested in
simplifications and clarifications than new features.

BUILDING AND RUNNING XV6

You will need a RISC-V "newlib" tool chain from
https://github.com/riscv/riscv-gnu-toolchain, and qemu compiled for
riscv64-softmmu.  Once they are installed, and in your shell
search path, you can run "make qemu".
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
$ wc README
48 336 2441 README
$
```

Fig. 3 — Rularea programelor native xv6: `cat README`, `echo`, `ls | grep c`, `wc README`.

Part 3 · Read the source

```
$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
```

(1) Apeluri de sistem folosite de user/cat.c și ce cer kernelului

`read(fd, buf, n)` — cere kernelului să copieze până la `n` octeți din fișierul/descriptorul `fd` într-un buffer din memoria procesului; `write(fd, buf, n)` — cere kernelului să scrie `n` octeți din buffer către descriptorul `fd` (de obicei stdout). `cat.c` rulează într-o buclă read→write până când `read()` întoarce 0. Ambele sunt declarate ca prototipuri de syscall în `user/user.h`, nu funcții obișnuite de bibliotecă.

(2) Unde e implementat sys_read?

`kernel/sysfile.c`, **linia 69** (`sys_write` imediat după, la linia 83). Aici kernelul validează descriptorul primit de la procesul user, găsește structura de fișier corespunzătoare și copiază efectiv datele.

(3) Diferența dintre codul din kernel/ și codul din user/

`kernel/` rulează în modul privilegiat al procesorului, cu acces direct la hardware — gestionează memoria, scheduler-ul, sistemul de fișiere și răspunde la syscall-uri. `user/` conține programe obișnuite (`cat`, `ls`, shell-ul) care rulează neprivilegiat; orice acțiune reală (citit un fișier, alocat memorie) trece printr-un syscall — o cerere explicită către kernel.

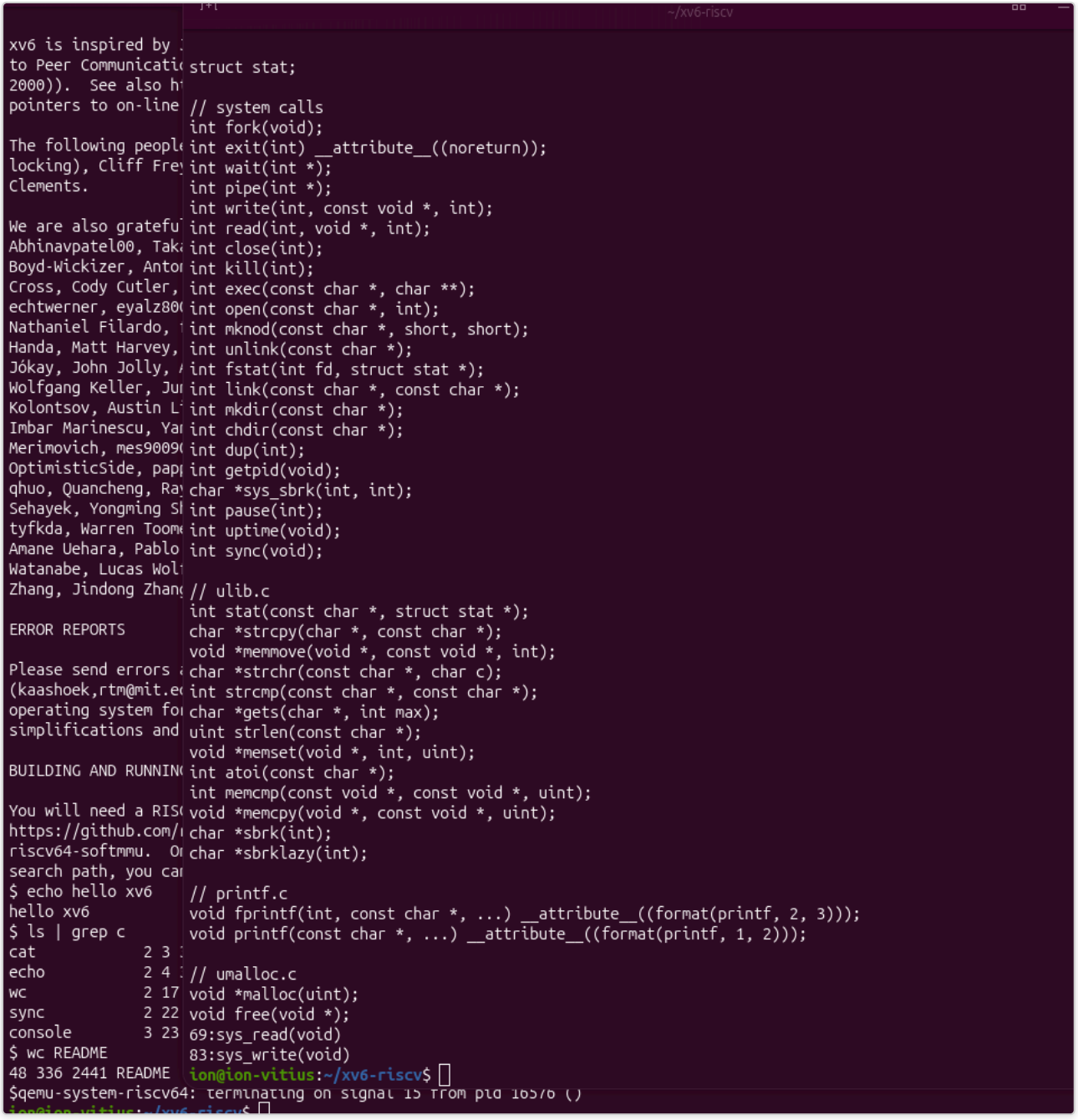


Fig. 4 — `cat user/user.h` (lista completă de syscall-uri disponibile programelor) și `grep -n` pentru localizarea `sys_read` / `sys_write` în kernel.

> **Observație:** lista de syscall-uri din `user/user.h` conține `pause(int)`, nu `sleep(int)` — copia de curs a xv6 a redenumit acest syscall. Detaliul a devenit relevant la Part 4.

Part 4 · Write your first xv6 program

Programul nou, `sleep`, pauzează execuția pentru un număr de ticks, apelând syscall-ul corespunzător din kernel.

```
// user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // syscall-ul se numeste "pause" in aceasta copie de
xv6
  exit(0);
}
```

> **Depanare:** compilarea a eşuat iniţial cu `implicit declaration of function 'sleep'`, deoarece syscall-ul nu se numeşte `sleep` în acest fork de curs, ci `pause` (confirmat la Part 3, în `user/user.h`). Fix: apelul din cod a fost schimbat din `sleep(...)` în `pause(...)`; numele programului (`sleep.c`, `UPROGS`) a rămas neschimbat.

Adăugarea programului nou în lista de build din Makefile:

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        ...
        $U/_sync\
        $U/_sleep
```

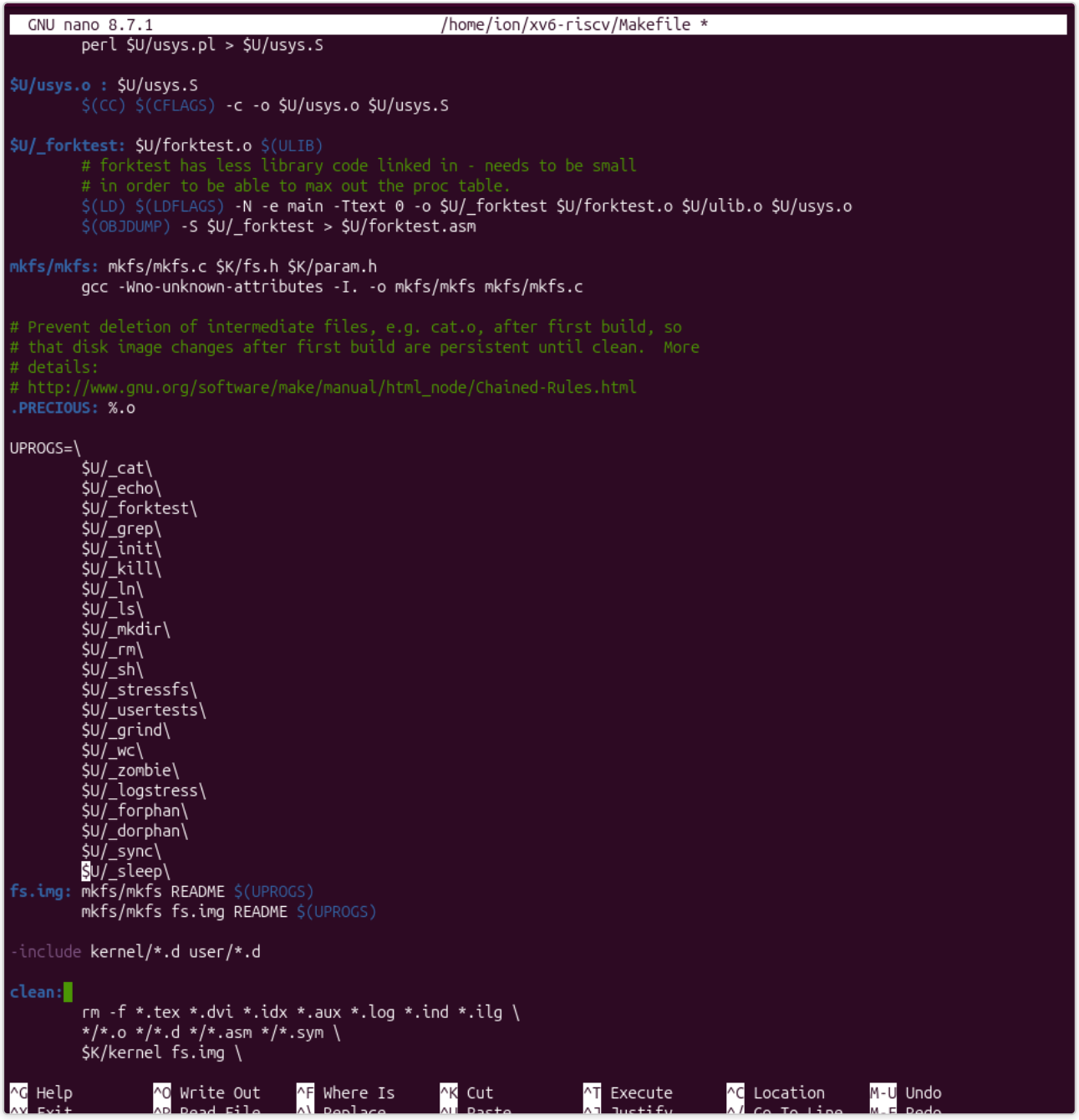


Fig. 5 — Lista `UPROGS` din Makefile, cu `$U/_sleep` adăugat la final.

Recompilare, boot și testare în xv6:

```
$ make qemu
...
$ sleep 10
$ sleep
usage: sleep <ticks>
```

`sleep 10` a pauzat execuția și a revenit la prompt după interval; `sleep` fără argument a afișat corect mesajul de utilizare — ambele comportamente cerute de fișă.

```
// umalloc.c
void *malloc(uint);
void free(void *);
69:sys_read(void)
83:sys_write(void)
ion@ion-vitius:~/xv6-riscv$ nano ~/xv6-riscv/user/sleep.c
ion@ion-vitius:~/xv6-riscv$ nano ~/xv6-riscv/Makefile
ion@ion-vitius:~/xv6-riscv$ cd ~/xv6-riscv
ion@ion-vitius:~/xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-m
ap=/home/ion/xv6-riscv=. -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builti
n-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-b
uiltin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin
-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-prote
ctor -fno-pie -no-pie   -c -o user/sleep.o user/sleep.c
user/sleep.c: In function ‘main’:
user/sleep.c:11:3: error: implicit declaration of function ‘sleep’ [-Wimplicit-function-declaration]
   11 |   sleep(atoi(argv[1]));
      |   ^~~~~
At top level:
cc1: note: unrecognized command-line option ‘-Wno-unknown-attributes’ may have been intended to silence earlier diagno
stics
make: *** [<builtin>: user/sleep.o] Error 1
ion@ion-vitius:~/xv6-riscv$ nano ~/xv6-riscv/user/sleep.c
ion@ion-vitius:~/xv6-riscv$ cd ~/xv6-riscv
make qemu
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-m
ap=/home/ion/xv6-riscv=. -march=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builti
n-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-b
uiltin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin
-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-prote
ctor -fno-pie -no-pie   -c -o user/sleep.o user/sleep.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_sleep user/sleep.o user/ulib.o user/usys.o user/pr
intf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_sleep > user/sleep.asm
riscv64-linux-gnu-objdump -t user/_sleep | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/sleep.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_m
kdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan
user/_dorphan user/_sync user/_sleep
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1019 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force
-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

Fig. 6 — Recompilare reușită și testarea programului: `sleep 10` și `sleep` (fără argument).

Livrabil și punctaj

Fișiere trimise prin git: `user/sleep.c` (programul funcțional) și `notes.md` (răspunsurile de la Part 2 și Part 3).

Criteriu	Punctaj
xv6 se compilează și pornește; a fost folosit și Part 2 e răspuns	3
Răspunsuri la citirea surselor (kernel/ vs user/, syscall-uri)	3
`sleep.c` funcționează: verificare usage corectă și apelează syscall-ul de pauzare	4

Referat întocmit pe baza lucrului efectiv realizat în terminalul Ubuntu (host `ion-vitius`), Operating Systems · FCIM/FAF · UTM · 2026–2027.